

Search

❖ الفرق بين abstract & interface :

1- التجريد abstract :

وهو عبارة عن كلاس يحتوي على خصائص وميثود مشتركة بين الكلاسات وكمكان جواه ميثود تاني كل كلاس يحط فيها **body** بناءا على الوظيفة اللي هيعملها وطبعا مقدرش اني اعمل منه **object** بيحتوي على خصائص وميثود عادية وكمكان ميثود بدون **body** تستطيع كلاسات تانية تورث منه وتنفذ ميثود **abstract** كل كلاس لما يورث منه يحط فيها **body** بناءا على الوظيفة اللي هيقوم بها بمعنى اصح اقله خد استخدم كل خصائص وميثود مشتركة ما بينا ده + كمل على ميثود تانية بناءا على وظيفتك وطبعا لازم تستخدم ميثود ده (مثال واقعي زي مثلا عقد بيع بيكون مكتوب في كل بنود وكل شي اللي اتفقتوا عليها فاضل ان المشتري يكمل بياناته في العقد زي اسمه الخ هو ده **abstract**).

```
1  <?php
2
3  abstract class Product
4  {
5
6      public $name;
7      public $price;
8
9      public function __construct($name, $price)
10     {
11         $this->name = $name;
12         $this->price = $price;
13     }
14
15     // ميثود مشتركة
16     public function getPrice()
17     {
18         return $this->price;
19     }
20
21     // لازم كل منتج ينفذها بطريقته
22     abstract public function calcDiscount();
23 }
24
25
26 class Book extends Product
27 {
28
29     public function calcDiscount()
30     {
31         return $this->price * 0.10;
32     }
33 }
34 class BabyCar extends Product
35 {
36
37     public function calcDiscount()
38     {
39         return $this->price * 0.05;
40     }
41 }
42
```

2- الواجهة interface :

هونوع من أنواع **abstraction** عبارة عن واجهة او **contract** تحتوى على مجموعة من القوانين اللى من خلال بجبر اى حد يستخدمنى لازم يستخدمهم والقوانين ده عبارة عن ميثود بدون **body** ولا تحتوى على خصائص ولا ميثود فيها **body** لكنه يمكن أن يحتوي على **Constants** وكمان يستطيع الكلاس الواحد انه يعمل **implements** لاكثر من **interface** (مثال واقعى زى لو هتقدم فى وظيفة وبيكون مطلوب منك اوراق معينة عشان تقدم فى الوظيفة وطبعا لازم تجيب كل الاوراق المطلوبة منك للتقديم فى الوظيفة هو ده الانترفيس)

```
1  <?php
2
3  interface Shippable
4  {
5
6      public function ship();
7      public function getWeight();
8  }
9
10 class Book implements Shippable
11 {
12
13     public function ship()
14     {
15         echo "Shipping Book via standard mail";
16     }
17
18     public function getWeight()
19     {
20         return 1;
21     }
22 }
23
24 class BabyCar implements Shippable
25 {
26
27     public function ship()
28     {
29         echo "Shipping BabyCar via freight";
30     }
31
32     public function getWeight()
33     {
34         return 15;
35     }
36 }
37
```

❖ تعدد الأوجه Polymorphism :

وهو تعدد الأوجه وهو عبارة عن مبدأ (Principle) في oop ومن خلاله تقدر تستخدم نفس اسم الدالة ولكن كل كلاس ينفذها بسلوك أو بشكل مختلف وله عدة طريق بـ يتحقق من خلالها زى :

1- الوراثة انك تستخدم نفس الميثود في كلاس تانى وتعملها Method Overriding

1- التجريد abstract

2- الواجهة interface

```
1  <?php
2  class Product
3  {
4
5      public $name;
6      public $price;
7
8      public function __construct($name, $price)
9      {
10         $this->name = $name;
11         $this->price = $price;
12     }
13
14
15
16     public function calcDiscount()
17     {
18         return $this->price;
19     }
20 }
21
22 class Book extends Product
23 {
24
25     public function calcDiscount()
26     {
27         return $this->price * 0.10;
28     }
29 }
30
31 class BabyCar extends Product
32 {
33
34     public function calcDiscount()
35     {
36         return $this->price * 0.05;
37     }
38 }
39
40
41 $book = new Book("PHP Book", 100);
42 $babyCar = new BabyCar("Car", 200);
43 $book->calcDiscount();
44 $babyCar->calcDiscount();
45
```